PhishLab

Platforma Cybersecurity - Kontrolowane środowisko edukacyjne

Koncepcja Projektu

Profil Grupy i Role (1/2)

Projekt realizowany jest przez czteroosobowy zespół. Role zdefiniowano w sposób zapewniający pokrycie warstwy serwerowej, klienckiej, infrastruktury oraz kontroli jakości oprogramowania.

- **Paweł Gabrysiak — Backend Developer / API Engineer**
  Odpowiedzialny za architekturę serwerową oraz implementację logiki biznesowej. Tworzy endpointy REST API niezbędne do zarządzania kampaniami oraz śledzenia metryk kliknięć. Integruje usługi w oparciu o framework Python (Django/Flask).
- **Maciek — Frontend Developer**
  Zajmuje się tworzeniem interfejsu użytkownika (UI) oraz logiką po stronie klienta. Implementuje widoki panelu administratora (Dashboard, kreator kampanii) oraz responsywne, interaktywne strony edukacyjne kierowane do użytkowników końcowych.

Profil Grupy i Role (2/2)

- **Mateusz — DevOps, Data & Full-stack**
  Odpowiada za projektowanie struktury relacyjnej bazy danych oraz konfigurację środowiska wdrożeniowego (konteneryzacja Docker). Zapewnia stabilność infrastruktury lokalnej i testowej, wspierając również prace nad modułami edukacyjnymi.
- **Paweł Handwerkier — QA & Integration Lead**
  Główny tester i koordynator integracji. Odpowiada za łączenie warstwy frontendowej z backendową. Weryfikuje kontrakty API, dba o integralność przesyłanych danych, przeprowadza testy interfejsu oraz monitoruje spójność i jakość całego systemu przed wdrożeniem.

Koncepcja aplikacji

PhishLab nie jest systemem antywirusowym.
Jest to dedykowane, kontrolowane środowisko edukacyjno-testowe pozwalające organizacjom na weryfikację świadomości cybernetycznej ich członków.

Zasada działania:

1. Administrator konfiguruje i uruchamia symulowaną kampanię phishingową.
2. Użytkownicy otrzymują spreparowane, bezpieczne wiadomości e-mail.
3. System gromadzi w tle anonimowe metryki reakcji.
4. Użytkownik, który podejmie niebezpieczną akcję (np. kliknie link), zostaje przekierowany na stronę uświadamiającą o błędzie.

Spis funkcjonalności

1. **Panel Administratora (Dashboard)**:
    - Konfiguracja kampanii (szablon, grupa docelowa, harmonogram).
    - Risk Score oceniający ogólny poziom bezpieczeństwa grupy.
    - Statystyki identyfikujące obszary wymagające dodatkowych szkoleń.
2. **Silnik wysyłkowy i Tracking (Endpointy)**:
    - Wysyłka symulowanych wiadomości.
    - Śledzenie akcji (otwarcia wiadomości, kliknięcia w odnośniki, czas reakcji).
3. **Moduł Edukacyjny**:
    - Dedykowane strony tłumaczące mechanizmy socjotechniczne wykorzystane w danej wiadomości.
    - Moduł weryfikacji wiedzy (krótkie quizy).

Stos technologiczny

Dobór technologii podyktowany jest wymogiem wydajności, łatwości skalowania oraz separacji logiki API od interfejsu.

- **Backend / API (Paweł G.):** Python (Flask / Django). Pozwala na szybkie dostarczenie stabilnych i bezpiecznych endpointów REST.
- **Frontend (Maciek):** React lub Vue.js. Zapewnia asynchroniczne odświeżanie danych i obsługę zaawansowanych wykresów analitycznych.
- **Infrastruktura (Mateusz):** PostgreSQL / SQLite. System skonteneryzowany za pomocą narzędzia Docker.
- **QA i Integracja (Paweł H.):** Narzędzia do testowania API (np. Postman, Swagger), narzędzia E2E (np. Cypress, Playwright) oraz manualna weryfikacja kontraktów pomiędzy warstwami.

Plan pracy (Tygodnie / Sprinty)

Sprint	Paweł G. (Backend)	Maciek (Frontend)	Mateusz (DevOps/DB)	Paweł H. (QA/Integracja)
Tydz. 1	Setup projektu, szkielet API	Repozytorium UI, makiety	Schemat bazy, konfiguracja Dockera	Definicja kontraktów API, plan testów
Tydz. 2	Endpointy: autoryzacja, kampanie	Logowanie, kreator kampanii	Modele bazy, skrypty migracyjne	Testy API autoryzacji, weryfikacja makiet
Tydz. 3	Endpointy: Tracking kliknięć	Dashboard (wykresy, tabele)	Połączenie DB z serwerem, CI/CD	Integracja widoków z API, testy trackingu
Tydz. 4	Endpointy: Moduł edukacyjny	UI stron edukacyjnych i quizów	Wsparcie API testów	Automatyczne i manualne testy E2E
Tydz. 5	Testy integracyjne, optymalizacja	Poprawki UX/UI, spięcie całości	Wdrożenie testowe, bezpieczeństwo	Testy regresyjne, akceptacja końcowa

Strategia pracy i wdrażania

- **System kontroli wersji:** Git. Praca w modelu Feature Branch Workflow.
- **Code Review:** Kod wprowadzany jest do głównej gałęzi repozytorium wyłącznie po wcześniejszej weryfikacji (Pull Requests).
- **Równoległy Development:** Ścisłe zdefiniowanie kontraktów API pozwala na niezależne i jednoczesne rozwijanie warstwy frontendowej i backendowej.
- **Kontrola Jakości:** Dedykowana rola QA zapewnia, że ewentualne rozbieżności między API a interfejsem wyłapywane są na bieżąco, co minimalizuje dług technologiczny przy końcowej integracji.

Analiza ryzyka i środki zaradcze

Ryzyko	Prawdopodobieństwo	Środek zaradczy
Blokowanie maili przez filtry antyspamowe	Wysokie	Skonfigurowanie testowej subdomeny z odpowiednimi rekordami DNS (SPF, DKIM).
Brak spójności między API a UI	Średnie	Ścisła weryfikacja dokumentacji i kontraktów API przez zespół QA od pierwszego sprintu.
Wyciek lub naruszenie prywatności danych	Niskie	Wymuszona anonimizacja metryk (hashowanie identyfikatorów), brak realnych danych osobowych w środowisku deweloperskim.
Problemy konfiguracyjne infrastruktury	Średnie	Wdrożenie Dockera zapewniające powtarzalność środowisk; fallback do lokalnej bazy SQLite.

Dziękujemy za uwagę

PhishLab – Tworzymy świadomość, nie oprogramowanie antywirusowe.